

HTA Firmware Tracker — README

A web dashboard that tracks the latest firmware/software versions for HTA's audio gear, so nobody has to manually check manufacturer sites.

What it does

- Tracks 47 pieces of equipment across DiGiCo, Yamaha, Solid State Logic, Allen & Heath, Shure, d&b audiotechnik, Dante/Audinate, and Waves.
- **43 items are checked automatically** — most by scraping a static page, four manufacturers (SSL, Allen & Heath, d&b, Dante) via a real headless browser where the site needs JavaScript or blocks plain requests.
- **4 items are flagged "Manual check"** with a direct link to where to look, because no version page/API could be found for them at all (see What's automatic vs. manual).
- Runs a background job that re-checks everything on an interval (default 24h) and highlights anything whose version changed since the last check.
- Dashboard grouped by manufacturer, with status badges (Up to date / Update available / Manual check / Check failed), per-item "Check" button, and an "Acknowledge" button to clear an update flag once you've dealt with it.

Quick start

```
cd "hta-firmware-tracker"
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
playwright install chromium
uvicorn app.main:app --reload
```

Open `http://127.0.0.1:8000` — the database (SQLite, at `data/tracker.db`) is created and seeded automatically on first run, and a check kicks off ~5 seconds after startup. (The `playwright install chromium` step downloads a ~90MB headless browser used only by the Allen & Heath / SSL / d&b / Dante checkers — see below.)

What's automatic vs. manual

Manufacturer	Method	Notes
DiGiCo	Scraped	<code>support.digico.biz</code> 's Zendesk help-center JSON API
Yamaha	Scraped	Static per-product firmware pages on <code>usa.yamaha.com</code>

Manufacturer	Method	Notes
Solid State Logic	Scraped	support.solidstatelogic.com's Zendesk API too; tracked via SOLSA (versions 1:1 with SSL Live console software) and the Network I/O firmware bundle
Allen & Heath	Scraped (headless)	Site returns HTTP 403 to plain requests (bot protection) — works fine in a real headless browser
Dante/Audinate	Scraped (headless)	The version table is inside a click-to-expand accordion, so it's not in the page until JS runs
d&b audiotechnik	Scraped (headless)	Their Download Center is a JS-driven search box behind a cookie-consent overlay; D40/D90 share one firmware release. DN1 Switch stays manual — no firmware entry exists for it in their system at all
Shure	Partial	Most receivers/transmitters/chargers are scraped from shure.com's static firmware-archive pages
Waves	Partial	Waves Central is scraped; SuperRack, WSG-HY128, and Waves Plugins stay manual — SuperRack installs through Waves Central (no standalone version page), and no dedicated page was found for the other two

Every manual row on the dashboard links straight to the page to check.

How the headless-browser checkers work

app/checkers/browser_base.py holds a small Playwright helper (browser_page() launches a headless Chromium tab, dismiss_cookie_banner() clicks past consent overlays). allenheath.py, dante.py, and dbaudio.py use it to load a page, interact with it if needed (click an accordion, type into a search box), and pull the version out of the rendered text. They're slower than the plain-HTTP checkers (each spins up a real browser) but run fine as part of the once-a-day background check. To add another one, copy the shape of dante.py, then register it in app/checkers/registry.py and point the relevant app/seed.py rows at it with "scrape" + a checker_key.

Sharing this with coworkers

Right now this runs as a normal FastAPI web app — anyone who can reach the host and port can view the dashboard, no separate frontend build needed. Options, roughly in order of effort:

- 1 **Run it on a machine that's always on** (a spare Mac mini, a NAS, a cheap VPS) and have coworkers hit `http://<that-machine>:8000`. This is also what makes automatic checking actually automatic — see below.
- 2 **Put it behind a reverse proxy with a real domain** (nginx/Caddy + Tailscale or a proper DNS record) once more than a couple people are using it.
- 3 **Slack integration** (the current next step): the scraping/DB layer is already separate from the web routes, so a Slack bot can call the same `app/runner.py` functions and post to a channel when `status == "update_detected"` — no rework needed.

Does it update automatically?

Yes, with one caveat: the background scheduler (`app/scheduler.py`, using `APScheduler`) only runs **while the app's process is alive**. If it's just running on your laptop, it only checks while your laptop has it open. For true "set and forget" automatic checking, run it on a machine that stays on (see above) — then it checks every `CHECK_INTERVAL_HOURS` (default 24, override via env var) with no one needing to touch it.

When a version changes, the item flips to an "Update available" badge and stays that way (even across further checks) until someone clicks "Acknowledge" — so a change can't get silently missed between visits.

Project layout

app/	
main.py	FastAPI routes + dashboard
models.py	Equipment / CheckLog tables (SQLAlchemy)
runner.py	Runs checkers, diffs versions, writes history
scheduler.py	Background interval job
seed.py	The 47-item equipment list + which checker covers each
checkers/	
digico.py, yamaha.py, waves.py, shure.py, ssl.py	Static-page scrapers
allenheath.py, dante.py, dbaudio.py	Headless-browser scrapers
browser_base.py	Shared Playwright helper
registry.py	Routes a checker_key to its module
templates/, static/	Dashboard UI
data/tracker.db	SQLite database (created on first run)

Adding equipment

Add a row to the EQUIPMENT list in `app/seed.py` (manufacturer, model, "scrape" or "manual", checker_key, source URL, optional notes), then restart the app (or run `python3 -m app.seed`) — `seed()` fully syncs the database to match the list: new rows are added, changed rows are updated, and rows removed from the list are deleted.